Connection Pooling & Keep-Alive

pool sizing, leaks and idle timeouts

Watch the 3-min video

In one line: opening a connection costs DNS, TCP and TLS round trips, so keep connections open and share them through a pool; size it with Little's Law, and watch idle timeouts and leaks.

1 Why new connections are expensive

Your FastAPI service calls a payments API on every request. Under load, p99 latency jumps and connection errors appear. The culprit is often one line: **creating a new HTTP client inside every request handler**.

Every new HTTPS connection has a setup cost: a **DNS lookup**, a **TCP handshake** and a **TLS handshake**. That is two or three round trips before the first byte of your request. At 50 ms per round trip, that's over **100 ms of pure overhead**, plus CPU for the cryptography on both sides.

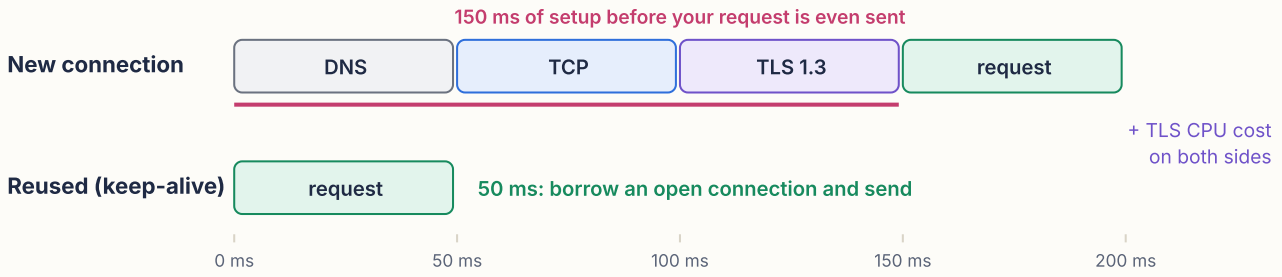


Figure 1. At 50 ms RTT a fresh connection spends ~150 ms on setup (DNS may be cached; TLS 1.2 adds a round trip). A reused one goes straight to the request.

2 Keep-alive and the connection pool

Keep-alive leaves the connection open after a response, so the next request can reuse it (the default in HTTP/1.1; HTTP/2 goes further and runs many requests over one connection). A **connection pool** manages a set of these open connections: you **borrow** an idle one, use it, and **give it back**. In Python that is `requests.Session` or an `httpx` client; database drivers and SQLAlchemy have their own pools. Without one, every call pays the full handshake again.

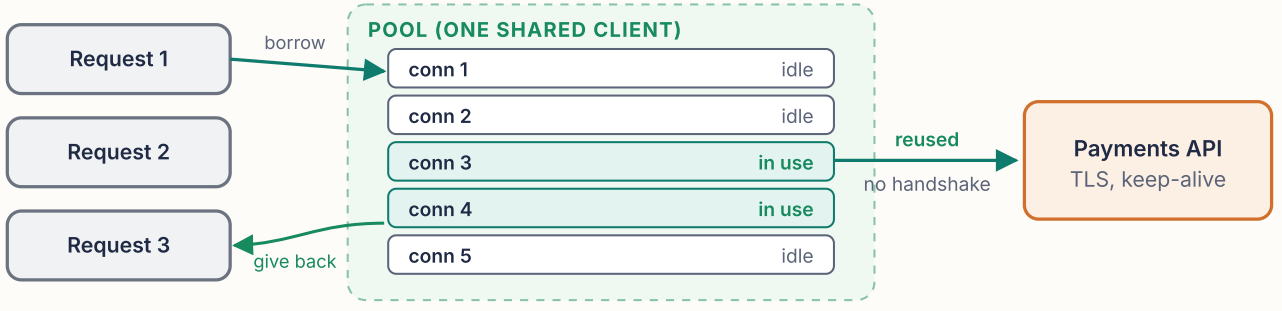


Figure 2. Requests borrow an idle connection, use it and return it. The pool keeps connections open, so calls skip the handshake.

Example: one shared client in FastAPI

Bad: a new `AsyncClient` inside the handler opens and tears down fresh connections on every request.

```
@app.get("/pay")
async def pay():
    async with httpx.AsyncClient() as c:   # new pool per request
        return (await c.post(URL, json=body)).json()
```

Fix: create one shared client at startup in FastAPI's `lifespan`, reuse it in every handler, and close it on shutdown.

```
@asynccontextmanager
async def lifespan(app):
    app.state.http = httpx.AsyncClient(timeout=5.0,
        limits=httpx.Limits(max_connections=30))
    yield
    await app.state.http.aclose()           # close on shutdown
app = FastAPI(lifespan=lifespan)
@app.get("/pay")
async def pay(request: Request):
    r = await request.app.state.http.post(URL, json=body)   # reused
    return r.json()
```

3 How big should the pool be?

Use **Little's Law**: connections needed = request rate × call duration.

```
200 calls/s × 0.1 s = 20 connections  (+ headroom, per worker or pod)
```

Too small, and requests queue for a free connection: latency climbs, then pool timeout errors appear, even though the downstream is perfectly healthy. **Too large**, and you overwhelm the downstream; big pools also waste memory and file descriptors.

WORKED EXAMPLE

Your service makes 200 payment calls per second, each taking 100 ms: about **20** connections busy at once. With ~50% headroom, aim for 30 in total. If that traffic is spread over 3 pods with 2 workers each (6 pools), each pool needs about 30 / 6 = **5**. Now a database with `max_connections = 100`, and 10 pods each holding a pool of 50 means up to **500**. The database starts refusing connections. Size each pool so the total across all pods fits, or put a pooler such as PgBouncer in front.

4 Two more traps: idle timeouts and leaks

Idle timeouts. If the server or load balancer closes idle connections after 60 s, but your pool keeps them for 90 s, you will reuse dead connections and see random resets. Keep the client's idle timeout **shorter** than the server's.

Leaks. If responses or streams aren't closed, their connections never return to the pool, until it is exhausted and every call waits for a pool timeout. Always use context managers (`async with client.stream(...)`, `with Session() as s`) so connections go back even on errors.

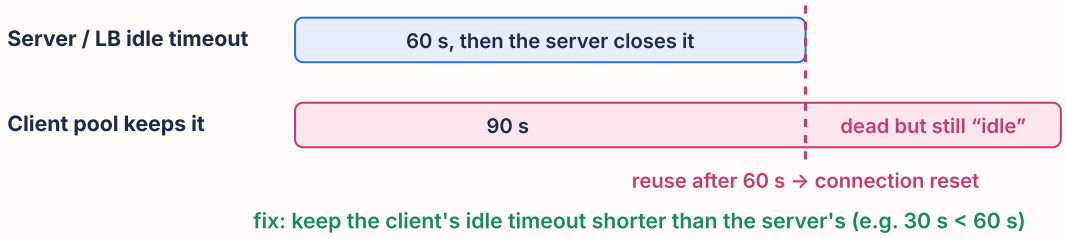


Figure 3. The server closed the connection at 60 s, but the client still thinks it is idle and usable. The next request on it fails.

COMMON MISTAKES

- A new client (or a new database engine) per request.
- Sizing each pod's pool on its own and forgetting to multiply by pods and workers.
- Raising the pool when the real problem is a leak or a slow downstream.

5 Key terms

Keep-alive: leave a connection open for reuse

Pool timeout: max wait to borrow a connection

Connection leak: borrowed, never returned

Connection pool: set of open connections to borrow

Idle timeout: close a connection unused this long

PgBouncer: external pooler in front of Postgres

6 Interview questions

Q1. Why reuse connections? BASIC

To skip DNS, TCP and TLS setup on every call. At 50 ms RTT that is about 100–150 ms and real CPU per call saved, which cuts latency and load on both sides.

Q2. What goes wrong when a pool is too small or too large? BASIC

Too small: requests queue for a connection and hit pool timeouts while the downstream is healthy. Too large: you exceed downstream limits like database max connections, and waste memory and file descriptors.

Q3. p99 jumped after traffic doubled, and logs show pool timeouts, but the payments API is healthy. What happened? SCENARIO

In flight doubled (say 20 → 40) and the pool stayed at 20, so requests wait for a connection. Raise the pool to rate × duration + headroom, and check for leaks (connections not returned).

Q4. You see random `connection reset` errors, mostly after quiet periods. Cause? SCENARIO

The pool reuses connections the server or load balancer already closed as idle (say at 60 s) while the client keeps them for 90 s. Set the client's keepalive expiry below the server's, e.g. 30 s.

Q5. Does HTTP/2 remove the need for a pool? TRICKY

Mostly for HTTP: one HTTP/2 connection multiplexes many concurrent requests, so you need few connections. You still reuse them, and database connections still need a pool.

Q6. The pool is exhausted at low traffic. Why? TRICKY

A leak: responses or streams not closed. Connections never return, so a small rate still drains it. Use context managers and watch the in-use count.

Next: SD-B-020 · Network Timeouts backendroadmap.com